

Topic: Instruction Interpretation and Sequencing — Self-Learning

Q. Explain the concepts of Instruction Interpretation and Sequencing. How is the control flow of instructions maintained during execution?

📌 Definition to Remember

Program execution relies on two mechanisms: **Instruction Interpretation** (the Control Unit decoding the instruction to understand *what* to do) and **Instruction Sequencing** (the process of using the Program Counter to determine *which* instruction to execute next).

1. Instruction Interpretation

The process where the Control Unit determines the meaning of a fetched machine instruction.

- * **Decoding:** The instruction in the Instruction Register (IR) is split into Opcode and Operands.
- * **Action:** The Opcode passes through a decoder circuit. This tells the Control Unit which specific hardware control signals to activate to execute the command (e.g., configuring the ALU for Addition).

2. Instruction Sequencing

The process of determining the memory address of the *next* instruction to fetch.

- * **The Program Counter (PC):** The PC holds the address of the next instruction.
- * **Sequential Flow:** Instructions are generally stored in sequential order. After an instruction is fetched, the CPU automatically increments the PC ($PC = PC + 1$ or $PC = PC + 4$) to point to the next instruction in memory.

3. Modifying Control Flow

While the default flow is sequential, programs require loops and decisions. **Control Flow Instructions** alter the sequencing process by changing the PC:

- * **Unconditional Branch (Jump):** Forces the PC to update to a new address regardless of conditions.
- * **Conditional Branch:** Evaluates a condition (using Status Flags, like Zero or Carry). If True, the PC updates to the branch address. If False, the PC remains sequential.
- * **Subroutines/Interrupts:** The current PC is saved (pushed to a stack) before jumping, so the CPU can later **Return** and resume sequential execution.

4. Fetch-Decode-Execute Cycle

The synthesis of both concepts:

1. **Fetch:** Read instruction at PC address. Sequence (increment) PC.
 2. **Decode (Interpret):** Analyze Opcode, determine hardware actions.
 3. **Execute:** Perform the operation; update PC if it's a branch instruction.
-

★ Must-Write Points (for 10 marks)

1. **Instruction Interpretation:** The Control Unit decodes the Opcode to understand the operation.
2. It generates specific hardware control signals based on the interpretation.
3. **Instruction Sequencing:** Determines the address of the next instruction to fetch.
4. Sequencing relies heavily on the **Program Counter (PC)**.
5. Default execution is **Sequential** (PC is automatically incremented after fetch).
6. Control flow is altered by **Branch Instructions** (Conditional or Unconditional) which manually overwrite the PC.
7. For subroutines, the old PC is pushed to a stack to maintain the eventual sequential return path.

⚡ Quick Recall

Interpretation (Decode Opcode, generate signals) → Sequencing (Find next address via PC) →
Default: $PC = PC + 1$ → Control Flow: Branches change the PC to jump/loop